Data Drift in a Seq2Seq Model for Robot Trajectory Prediction in the RoboCup SSL

Ivan G. Yamasaki[1][0009−0000−4776−134X], Marcos R. O. A. Máximo[1][0000−0003−2944−4476], and Paulo M. Tasinaffo[1][0000−0001−7069−8430]

Aeronautics Institute of Technology, São José dos Campos, Brazil

Abstract. Trajectory prediction models are normally reported on a single fixed split of the data, which says little about how long they stay useful. We take a sequence-to-sequence (seq2seq) predictor for the RoboCup Small Size League (SSL), trained on logs from the 2019 tournament, and evaluate it without any retraining on the tournaments of 2021 to 2025. The point of comparison is a baseline built on a Kalman filter for tracking and prediction, the classical choice in the league. Accuracy does fall as the league moves away from the training season. The Kalman-filter baseline falls faster, though, so the margin in favor of the learned model widens instead of closing. Importance weighting attributes 59–72 % of the extra error to a change in the input distribution rather than in the input–output relation, and points to tail acceleration as the main culprit; a classifier two-sample test agrees, season by season. We put that diagnosis to work twice: it tells us how to weight the replay buffer of a light fine-tuning, which then recovers accuracy with the least forgetting, and it yields a monitoring signal that needs no model evaluation and stays below 0.13 false alarms per thousand trajectories on the drift-free training season.

Keywords: trajectory prediction · data drift · covariate shift · drift detection · RoboCup SSL · mobile robots

1 Introduction

Predicting where a robot will be a fraction of a second from now is a routine requirement in systems with fast dynamics. In the RoboCup Small Size League (SSL), where six to eleven robots per team move at several meters per second under centralized control, such predictions feed planning, positioning and real-time decision making. All matches used here come from Division A, the top division of the league, where the most mature teams play and where hardware and playing styles change fastest.

The Kalman filter [3] is the traditional tool for this task, and it is hard to beat on cost: deterministic, cheap and easy to tune. We call *Kalman-filter baseline* the constant-velocity predictor given by its steady state, and use it throughout as the model-based reference. Its weakness is the assumption behind it, namely that the dynamics are smooth enough for a local linear model. Sequence-to-sequence

(seq2seq) networks drop that assumption and learn the patterns of future motion straight from logged matches; Steuernagel and Máximo [1] showed that such a model beats the kinematic reference in the SSL.

Being more accurate in one season, though, is not the same as still being useful three seasons later. The league moves fast. Robots are rebuilt with stronger motors, control stacks are rewritten, and playing styles change from one tournament to the next. A network that learned the motion patterns of an old season can decay silently under all of this, whereas a model-based predictor has no learned parameters and might be expected to survive untouched. That expectation is worth testing, and testing it is the data drift problem we study.

Our contribution is threefold: we measure how a seq2seq trajectory predictor for the SSL ages over seven years of official RoboCup logs, we identify the mechanism behind the loss, and we turn that mechanism into adaptation and monitoring procedures that can be deployed. Relative to [1], the question changes. It is no longer whether the learned model beats the kinematic reference on a fixed split, but how far that advantage survives as the data drift away from the training season. Four questions organize the study:

1. how the error evolves across seasons, at two prediction horizons, and how that evolution compares to the Kalman-filter baseline;
2. what fraction of the degradation is explained by covariate shift, and which kinematic dimension dominates it;
3. whether light adaptation recovers accuracy without catastrophic forgetting;
4. whether an online detector can flag the drift with a false-alarm rate low enough to trigger retraining.

The paper is organized as follows. Section 2 reviews related work and Section 3 describes the data, the model and the metric. Section 4 measures the degradation over time, and Section 5 splits it into covariate shift and concept drift. Sections 6 and 7 deal with the two responses, adaptation and online detection. Section 8 concludes.

2 Related Work

Trajectory prediction. Most progress on learned trajectory predictors has come from the pedestrian and driving domains. Social-LSTM [16] was among the first to model interaction explicitly, pooling the hidden states of neighboring LSTMs so that each agent is predicted with some awareness of the others. Trajectron++ [17] replaced that fixed pooling with a dynamic graph and added a latent variable, which produces several plausible futures rather than one. AgentFormer [18] went further and let a transformer attend over time and over agents at once, keeping social and temporal reasoning together. Diffusion models [19] have since been used to generate entire trajectory distributions by progressive denoising. In the SSL, Steuernagel and Máximo [1] applied an attention-based seq2seq model at the horizons we also use and reported a clear gain over a kinematic predictor; that model is our direct baseline. All of these evaluations rely

on a fixed split, with training and test data collected under the same conditions. Stability over time is simply not part of the protocol.

Data drift and its detection. Gama et al. [10] separate virtual drift, a change in $p(x)$, from real drift, a change in $p(y|x)$. The difference has practical consequences: virtual drift can in principle be corrected by reweighting data that is already on disk, while real drift demands fresh labels. Applied work often skips this step and reports only that accuracy dropped, although the remedy depends entirely on which component moved. Detectors come in two families. Error-based ones such as ADWIN [12] and Page–Hinkley [13] monitor the loss and therefore need labels; data-based ones such as KSWIN [14] watch the inputs and do not. Lu et al. [11] argue that a detector only becomes trustworthy in production once its false-alarm rate has been calibrated, and we adopt that requirement as a design constraint in Section 7. On the response side, adaptation is usually posed as a stability–plasticity trade-off, with elastic weight consolidation [15] as the standard regularizer against catastrophic forgetting.

Covariate shift. When only $p(x)$ moves, the classical correction is importance weighting. Old samples are reweighted by the ratio between the new and the old input density, so that a quantity computed on old data behaves as if it had come from the new one. Estimating the two densities separately is wasteful, and unstable already in moderate dimension, which is why several methods estimate the ratio directly: KLIEP [5] and least-squares importance fitting (LSIF) [7,6] are the best known. RuLSIF [8] fits a relative ratio instead, accepting a small bias in exchange for bounded weights and lower variance. The classifier two-sample test [9] offers an assumption-free alternative: train a discriminator to tell the two samples apart and read the shift off its accuracy. We use LSIF as the main estimator and the other two as robustness checks.

3 Data and Model

Data. We use the official logs of the 2019–2025 RoboCup tournaments, with 2020 missing because the pandemic cancelled the competition altogether. Six Division A matches were taken from each tournament — the same number every year, so that no season weighs more than another — for a total of 36 matches recorded at 60 Hz. The 2021 edition was played entirely in simulation, on grSim, since the pandemic still ruled out an in-person tournament. We keep it in the descriptive analyses, as it carries by far the largest distribution jump of the period (+3.05 mm of excess ADE), but part of that jump probably reflects the gap between simulation and reality rather than organic drift. It is therefore reported separately throughout, and left out of the adaptation experiments of Section 6: consolidating a model against an anomalous simulated season would contaminate the old regime with a distribution that no longer describes the league. Since every log comes from the same division, the growing error cannot be blamed on a change in the competitive level of the sample. The per-trajectory analyses draw

8 000 windows per match, which gives a temporally ordered stream of 288 000 trajectories, 48 000 per year. All six matches of 2019 go into the baseline, and for good reason: a sensitivity sweep puts ADE_{2019} anywhere between 4.53 and 6.20 mm if a single reference match is used, enough to flip the sign of the excess in four of the five remaining years.[1]

Model and benchmark. The predictor follows the architecture of [1]: a 128-unit LSTM encoder, the additive attention of Bahdanau et al. [2], and an autoregressive LSTM decoder without teacher forcing. Training uses Adam [4] with an initial learning rate of 10^{-3}, batch size 1024 and early stopping. Every input step carries six normalized variables: the position x, y and the velocity v_x, v_y of the robot in the horizontal plane of the field, together with $\sin\psi$ and $\cos\psi$, where ψ is the heading angle about the vertical axis. We fit the normalization once, on the 2019 season, and apply it unchanged to every later tournament; otherwise the later years would be measuring a change in preprocessing rather than in the data. One detail of the reference model matters a great deal here. The target lives in the space of normalized incremental velocities, and the future position is rebuilt from the last observed state, so what the network really predicts is accumulated dynamics. That makes it directly sensitive to the velocity and acceleration distributions. The Kalman-filter baseline is the constant-velocity predictor given by the steady state of a Kalman filter [3]: it holds the last observed velocity and extrapolates the position linearly. It is deterministic, has no tunable parameters and is identical in every season, so there is no training distribution for it to drift away from. Any error growth on its side measures how far the robots themselves have moved away from constant-velocity behavior.

Metric. For N trajectories and horizon H, the average displacement error is

$$\mathrm{ADE} = \frac{1}{NH} \sum_{i=1}^{N} \sum_{t=1}^{H} \left\| \hat{p}_{i,t} - p_{i,t} \right\|_2 , \tag{1}$$

with $\hat{p}_{i,t}$ and $p_{i,t}$ the predicted and the observed planar position of trajectory i at step t. The descriptive results cover two horizons, $30 \to 15$ and $60 \to 30$, where $a \to b$ means observing a past frames and predicting the next b — half a second of history for a quarter of a second of future, at 60 Hz. Everything that follows on mechanism and detection uses $30 \to 15$.

4 Temporal Degradation

Fig. 1 summarizes three descriptive findings.

The seq2seq model degrades outside its training distribution. ADE grows in every season after 2019, but two effects have to be read apart. The virtual 2021

[1] Pipeline code and processed data available from the authors upon request.

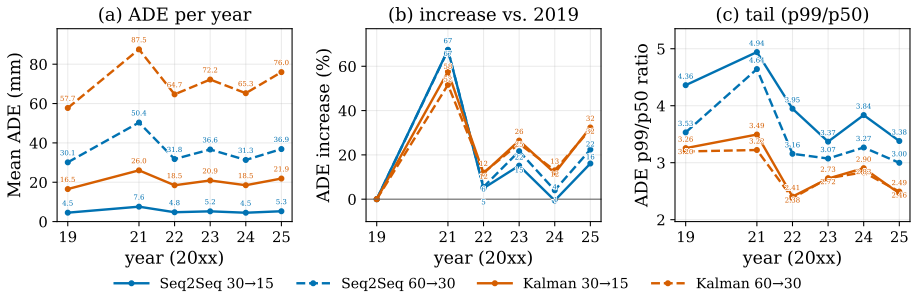

Fig. 1: Temporal degradation (color = model, line style = horizon). (a) Mean ADE per year. (b) Percentage increase in ADE relative to 2019. (c) p99/p50 ratio of the per-trajectory ADE: the 2021 jump is a tail effect (4.36 → 4.94), whereas in 2022–2025 the ratio falls below baseline while the median rises by about 25 %.

season stands alone, with a jump of +3.05 mm (+67 %) for the reason given in Section 3. Set that season aside and what remains over 2022–2025 is a slow upward trend against the same 2019 baseline: +0.23 mm in 2022, +0.73 mm in 2025 at the 30 → 15 horizon, with 2024 almost back to zero. These are two separate phenomena, not one stacked on the other: an abrupt regime change in a simulated tournament, and a gradual drift in the in-person ones. Both horizons show the same pattern.

The Kalman-filter baseline degrades more. Its relative error grows faster than the seq2seq model's in nearly every season. The baseline has no learned parameters, so what deteriorates is not the match to a training distribution but the match between the constant-velocity assumption and the way the robots now move. This does not make the neural model stable — its absolute error grows too. It means the kinematic reference is even more sensitive to the regime change, and the advantage of learning therefore widens under drift.

The new regime enters through the tail, then spreads. The p99/p50 ratio rises only at the 2021 jump (4.36 → 4.94), where the model fails disproportionately on a tail of trajectories. In the seasons that follow, the ratio falls below baseline, to 3.4–4.0, because the median climbs by about 25 % up to 2025 while the 99th percentile stays flat. Behavior that used to be rare has become ordinary, and the gradual degradation now sits in the body of the distribution.

Looking at the input side directly confirms the picture (Fig. 2). We use two distances between distributions, both computed per year against the 2019 baseline: the Kolmogorov–Smirnov statistic, which is the largest gap between the two cumulative distributions, and the Wasserstein distance, which is the average displacement needed to turn one into the other. Both panels are read the same way, a higher curve meaning a season further from 2019. Velocity

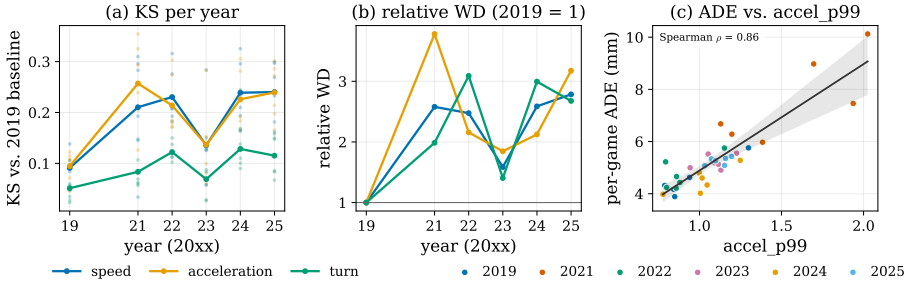

Fig. 2: Kinematic drift. (a) Kolmogorov–Smirnov statistic per year against the 2019 baseline (lines: annual mean; dots: individual matches); higher means a larger discrepancy between that year's distribution and 2019's. (b) Wasserstein distance indexed to the 2019 level, read the same way. (c) Mean ADE per match against the match's `accel_p99`, with a regression line and a 95 % bootstrap band.

and acceleration show clear and persistent shifts; turn rate moves much less. At match level, ADE grows almost monotonically with the tail acceleration of the match (`accel_p99`, Spearman $\rho = 0.86$). The more aggressive the match, the worse the model does, whatever season it belongs to.

5 How Much of It Is Covariate Shift?

The joint distribution factors as $p(x, y) = p(y|x)\,p(x)$ [10]. Covariate shift moves only $p(x)$ and can be undone by reweighting old samples, with no retraining at all; concept drift moves $p(y|x)$ and needs fresh labels. Telling the two apart therefore decides whether the cheap fix is available.

LSIF [5,7,6] estimates the density ratio $\hat{w}(x) = p_{\text{target}}(x)/p_{2019}(x)$ directly as a combination of Gaussian kernels, with coefficients $\boldsymbol{\alpha}$ obtained from

$$\min_{\boldsymbol{\alpha} \geq 0} \; \tfrac{1}{2}\,\mathbb{E}_{p_{2019}}[\hat{w}(x)^2] - \mathbb{E}_{p_{\text{target}}}[\hat{w}(x)] + \lambda \|\boldsymbol{\alpha}\|^2, \tag{2}$$

with $x \in \mathbb{R}^8$ collecting the kinematic features of each observed window: mean, p90 and p99 of speed and acceleration, plus mean and p90 of turn rate, clipped at the 99.5th percentile of the pool and passed through $\log(1 + \cdot)$. Stability is monitored by the effective sample size $\text{ESS} = (\sum_i \hat{w}_i)^2/(n \sum_i \hat{w}_i^2)$, with the usual $\text{ESS} > 0.5$ rule of thumb [6]. The weighted error and the recovered fraction of the excess are

$$\text{ADE}_{\text{IW}} = \frac{\sum_i \hat{w}(x_i)\, e_i}{\sum_i \hat{w}(x_i)}, \quad \text{rec.} = \frac{\text{ADE}_y - \text{ADE}_{\text{IW}}}{\text{ADE}_y - \text{ADE}_{2019}}, \tag{3}$$

where e_i is the ADE of trajectory i and ADE_y the unweighted mean error of season y. A recovery of 100 % would mean pure covariate shift, and 0 % pure concept drift.

Table 1: Importance-weighting decomposition (LSIF) with block bootstrap by match ($n_{\text{boot}} = 2000$, 6 blocks per year). Baseline $\text{ADE}_{2019} = 4.53\,\text{mm}$; corr. $= \text{ADE}_y - \text{ADE}_{\text{IW}}$. 2024 has negative excess and is excluded from the recovery column. All classifier tests give $p < 10^{-3}$ by permutation. †2022 has a small excess (0.23 mm), so the percentage should be read with care.

Year	ADE_y	ADE_{IW}	rec. (%) [95 % CI]	corr. (mm)	AUC
2021	7.58	5.70	61.6 [43.7; 79.3]	1.88 [1.33; 2.42]	0.754
2022	4.75	4.62	59.0 [9.2; 115.2]†	0.13 [0.02; 0.26]	0.647
2023	5.21	4.72	71.8 [43.8; 109.2]	0.49 [0.30; 0.75]	0.657
2024	4.50	4.24	—	—	0.612
2025	5.26	4.75	69.3 [57.2; 79.0]	0.51 [0.42; 0.58]	0.710

Robustness. Every number above depends on the weights behaving under the heavy tail of `accel_p99`, so we ran three checks. First, the weights stay bounded: $\max_i \hat{w}_i \in [2.9; 4.4]$ in every season, and the ESS of 0.58–0.73 comes from the near-zero mass given to the new regime, which is the right mechanism for reconstructing 2019, not from a handful of extreme weights (the top 1 % of trajectories carries only 3–4 % of the mass). Second, capping the weights at $c \in \{1.5, \ldots, 50\}$ leaves recovery saturated from about $c = 3$ onward. Third, the RuLSIF relative-ratio transform $\hat{w}/(\alpha\hat{w} + 1 - \alpha)$ with $\alpha = 0.1$ keeps recovery at 59–65 %. We also tried fitting RuLSIF directly and discarded it: in this configuration it returns almost uniform weights (ESS ≥ 0.94) and underestimates recovery at 9–23 %, which the two previous checks identify as an artifact of the fit rather than a property of the estimand.

Reading. Table 1 puts LSIF recovery at 59–72 % in the years with a positive excess, with confidence intervals that exclude zero. The absolute correction supports a stronger statement, since its lower bound stays positive even in 2022 ($\geq 0.02\,\text{mm}$): with only six bootstrap blocks, reweighting still moves the error toward the 2019 level by an amount distinguishable from zero. The classifier two-sample test agrees. It separates each season from 2019 with AUC $\in [0.61; 0.75]$ and ranks the seasons by severity in the same order, without estimating any density ratio. What is left unrecovered, 30–40 %, is residual concept drift plus sampling noise.

Refitting LSIF on one feature at a time makes the mechanism concrete (Table 2). Tail acceleration leads in every season, with `accel_p90` close behind; the speed features recover far less, and the turn-rate features are essentially inert, below 5 % throughout. The ranking barely changes from year to year. The league has adopted more aggressive acceleration profiles, and that is the main vector of degradation. The ESS column moves in the opposite direction, as expected: the features that shifted the most are the ones whose reweighting costs the most effective sample.

Table 2: Marginal recovery per feature (LSIF refitted on one feature at a time), for the three seasons with a significant excess.

Feature	2021 (%)	2023 (%)	2025 (%)	mean ESS
`accel_p99`	**57.1**	52.6	51.7	0.55
`accel_p90`	54.0	51.3	51.1	0.57
`accel_mean`	43.0	47.0	49.5	0.68
`speed_p99`	17.2	36.1	40.1	0.84
`speed_p90`	15.8	34.4	38.0	0.85
`speed_mean`	7.9	24.7	27.4	0.91
`turn_p90`	3.9	0.2	0.0	0.97
`turn_mean`	5.3	0.9	0.0	0.96

Table 3: Adaptation strategies under the common conservative protocol. The unadapted model scores 4.584 mm on the old regime and 4.965 mm on the new data. EWC reports the median of the $\lambda \in [50;\ 10^7]$ plateau. The last three rows are driven by the covariate diagnosis.

Strategy	`cat_ratio` ↓	ADE (mm)	
		old	new
Conservative FT ($r = 0$)	0.457	4.480	4.762
EWC (λ plateau)	0.457	4.481	4.817
Uniform replay ($r = 0.5$)	0.438	4.439	**4.760**
Importance-weighted replay	**0.417**	**4.404**	4.761
Acceleration-selective FT	0.467	4.526	4.811
Selective + IW replay	0.449	4.441	4.800

6 Adaptation

Since the excess is mostly covariate, we compared four families of adaptation strategies under a single conservative protocol: old regime = 2019, new data = 2022–2025 (500 trajectories per match, 2021 excluded as an outlier), frozen encoder, two trainable decoder layers, lr $= 10^{-5}$ and five epochs. Forgetting is measured by `cat_ratio`, the fraction of old-regime trajectories that get worse after the update. Table 3 collects the results.

Naive fine-tuning is destructive: without the conservative protocol, `cat_ratio` reaches 0.82 after a single epoch. Elastic weight consolidation [15] is the standard remedy and adds $\frac{\lambda}{2}\sum_i F_i(\theta_i - \theta_i^*)^2$, where θ_i is the i-th parameter, θ_i^* its value before the update, and F_i the corresponding entry of the diagonal Fisher information, which says how much the old task depends on that parameter. In our setting the Fisher term does nothing. Sweeping $\lambda \in \{0, 50, \ldots, 10^7\}$ leaves `cat_ratio` pinned near 0.46 across eight orders of magnitude, because the diagonal is numerically tiny ($F_i \approx 10^{-7}$), as one would expect from an already

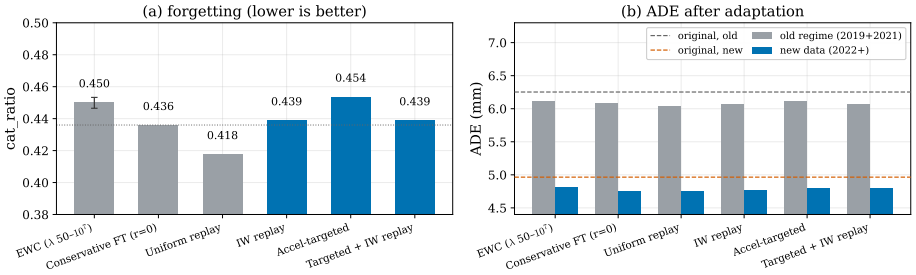

Fig. 3: Adaptation strategies under the same protocol (blue: variants driven by the covariate/acceleration diagnosis). (a) Forgetting, with the error bar showing the EWC λ plateau and the dotted line the no-replay control. (b) ADE after adaptation on both regimes, with the unadapted model as reference (dashed lines).

converged decoder. The old regime is protected by the small learning rate and the frozen encoder, not by the penalty.

Mixing a replay buffer of original data into the update, at a replay-to-new ratio $r = 0.5$, improves all three metrics at once: `cat_ratio` drops below the no-replay control and the ADE on new data is the lowest in the table. The gain is modest, since the conservative schedule already limits forgetting, but it is consistent and costs essentially nothing.

Steering the adaptation by the covariate diagnosis gives a more interesting split (Fig. 3). Sampling the replay buffer in proportion to $\hat{w}(x)$ helps: it reaches the lowest `cat_ratio` (0.417) and the lowest old-regime ADE in the table, albeit by a small margin and on a single seed. Hard selection, on the other hand, hurts. Fine-tuning restricted to the top 50 % of new data by tail acceleration — exactly the region where $p(x)$ moved — gives the worst forgetting of all six rows (0.467), worse even than the control. The tail is the subset farthest from the 2019 regime, so training on it alone displaces the weights more. The unchanged body of the distribution, which hard selection throws away, was acting as an implicit anchor; reweighting keeps that anchor and merely emphasizes the relevant tail. The diagnosis is thus worth using for monitoring, for explaining the excess and for weighting replay, but not for deciding which data to discard.

7 Online Detection

The operational question is whether drift can be flagged automatically on the 288 k-trajectory stream. We judge a detector by three quantities [11]: FAR_{2019}, the alarms per thousand trajectories in the training year, where no drift exists and every alarm is by definition false (admissibility ceiling 0.20/1k); coverage, the number of 2021–2025 seasons with at least one alarm; and latency after each year boundary. To avoid tuning and reporting on the same data, calibration runs under a two-fold cross-validation by matches: half the matches of each year select

Table 4: Layered detection architecture, with metrics on the test split of each fold (A/B). All signals use robust normalization with $w = 200$; each detector is configured per fold during calibration.

Layer	Signal	FAR/1k (A/B)	Cov.	Action
Primary	seq2seq ADE	0.042 / 0.125	5/5, 4/5	retrain
Watch	`accel_p95`	0.292 / 0.250	5/5, 5/5	monitor
Confirmation	AND(ADE, accel)	0.000 / 0.000	3/5, 2/5	confirm
Corroborator	Kalman-filter ADE	0.167 / 0.083	5/5, 5/5	redundancy

the ADWIN [12] configuration out of a 96-point grid, and the untouched half is evaluated once. Every figure below is out of sample.

The detector is only viable at all thanks to a local robust normalization of the per-trajectory signal, by median and MAD over a 200-trajectory window. On the raw signal, no configuration in the grid is admissible in either fold, and the 21 conventional smoothers we tried (moving averages, EWMA, Holt, Savitzky–Golay) all raise FAR by injecting the very autocorrelation that ADWIN assumes away. On the normalized ADE the winning family is stable across folds ($\delta = 10^{-4}$, windows 1000/5000): FAR_{2019} of 0.042/1k with 5/5 coverage in fold A, 0.125/1k with 4/5 in fold B, at a latency of 0.2 to 1.5 matches. The holdout does more than add confidence here, it changes what one would deploy: the configuration picked by in-sample selection keeps FAR ≈ 0 and then covers only 2–3 of the 5 seasons out of sample.

The diagnosis of Section 5 also buys a second monitoring channel. Since the shift is dominated by tail acceleration, the features of the observed window already carry the drift signal, with no need to run the model or to wait for the 15 future steps that ADE requires. Applying the same protocol to four kinematic signals reproduces the ranking of the per-feature decomposition, with `accel_p95` in front (FAR = 0.25–0.29/1k, 5/5 coverage in both folds).[2] No single feature is admissible on its own, because kinematics vary naturally within 2019 as well. Still, the acceleration channel fires before the error channel in 8 of 10 year-folds, which is enough for a watch layer. At the other extreme, requiring both alarms within a coincidence window of 200 trajectories drives observed false alarms to zero in both folds, at the cost of coverage (2–3 of 5).

Table 4 summarizes the resulting three-layer architecture. Two caveats and one negative result should be recorded. Per-trajectory FAR assumes roughly independent trajectories within 2019, and the serial correlation is certainly there ($\rho_1 = 0.54$); a permutation test suggests the violation is conservative, since shuffling trajectories within matches drives false alarms to zero in all 200 replicates. Among the alternatives, Page–Hinkley [13] aggregated over windows achieves FAR = 0 but catches only the abrupt 2021 jump, while KSWIN [14] is inadmis-

[2] p95 rather than the dominant p99, because the observed window holds only 29 acceleration samples.

sible in every configuration (FAR ≥ 0.438/1k). The negative result is structural. Combining detectors that run at different time scales simply does not work here: the closest pair of alarms is 214 trajectories apart, beyond any coincidence window one would accept. Aligning the scales first removes the problem, and that is how the confirmation layer above was built.

8 Conclusions and Future Work

A seq2seq trajectory predictor for the SSL loses accuracy as the league moves away from its training season. The loss is gradual, it is measurable, and most of it traces back to one specific change in the input distribution. Excess ADE trends upward over 2022–2025, separately from the one-off jump of the virtual 2021 edition, and the Kalman-filter baseline degrades faster still. Importance weighting recovers between 59 and 72 % of that excess, an estimate that survives weight clipping and the relative-ratio transform and is corroborated by an independent classifier test. Tail acceleration is the dominant vector.

On the response side, light adaptation keeps forgetting under control (`cat_ratio` from 0.82 to 0.46), EWC turns out to be inert over eight orders of magnitude of λ, and a replay buffer improves forgetting and adaptation at once, best of all when sampled by importance weights. Hard selection by the drifting tail backfires. On the operational side, ADWIN over a robustly normalized error stream reaches 0.04–0.13 false alarms per thousand trajectories out of sample, covering 9 of 10 year-folds, and the same diagnosis yields a model-free acceleration channel that anticipates the error alarm in 8 of them.

Several limitations bound these conclusions. Only Division A appears in the logs. Six matches per season keep the block confidence intervals wide, and the Wilson intervals of the holdout cross the admissibility ceiling, so a claim of admissibility at 95 % confidence would need more 2019 matches. ADE is single-mode and would have to give way to min-ADE$_K$ [17,18] for multimodal predictors. Robust normalization assumes gradual drift and would absorb part of an abrupt change. The residual concept-drift fraction is not isolated per feature, and it may hide covariate shift in dimensions we did not measure. The 2021 season, finally, was played in simulation, so its error jump mixes the gap between simulation and reality with seasonal drift; the conclusions about the gradual trend rest on 2022–2025 alone. The natural next step is a deeper adaptation, retraining the encoder itself with an importance-weighted replay buffer.

Acknowledgements

Ivan Yamasaki acknowledges the support of ITAEx and Nubank, provided within the framework of the ITA Undergraduate Research Program in Artificial Intelligence (2025–2026).

References

1. Steuernagel, L., Máximo, M. R. O. A.: Trajectory Prediction for SSL Robots Using Seq2seq Neural Networks. In: Eguchi, A., Lau, N., Paetzel-Prüsmann, M., Wanichanon, T. (eds.) RoboCup 2022: Robot World Cup XXV. LNCS, vol. 13561, pp. 27–38. Springer, Cham (2023). doi:10.1007/978-3-031-28469-4_4
2. Bahdanau, D., Cho, K., Bengio, Y.: Neural Machine Translation by Jointly Learning to Align and Translate. In: 3rd International Conference on Learning Representations (ICLR) (2015)
3. Kalman, R. E.: A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering **82**(1), 35–45 (1960)
4. Kingma, D. P., Ba, J.: Adam: A Method for Stochastic Optimization. In: 3rd International Conference on Learning Representations (ICLR) (2015)
5. Sugiyama, M., Nakajima, S., Kashima, H., von Bünau, P., Kawanabe, M.: Direct importance estimation with model selection and its application to covariate shift adaptation. In: Advances in Neural Information Processing Systems (NIPS) (2008)
6. Sugiyama, M., Kawanabe, M.: Machine Learning in Non-Stationary Environments: Introduction to Covariate Shift Adaptation. MIT Press (2012)
7. Kanamori, T., Hido, S., Sugiyama, M.: A least-squares approach to direct importance estimation. Journal of Machine Learning Research **10**, 1391–1445 (2009)
8. Yamada, M., Suzuki, T., Kanamori, T., Hachiya, H., Sugiyama, M.: Relative density-ratio estimation for robust distribution comparison. Neural Computation **25**(5), 1324–1370 (2013). doi:10.1162/NECO_a_00442
9. Lopez-Paz, D., Oquab, M.: Revisiting Classifier Two-Sample Tests. In: 5th International Conference on Learning Representations (ICLR) (2017). arXiv:1610.06545
10. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4), 1–37 (2014). doi:10.1145/2523813
11. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under concept drift: a review. IEEE Transactions on Knowledge and Data Engineering **31**(12), 2346–2363 (2019). doi:10.1109/TKDE.2018.2876857
12. Bifet, A., Gavaldà, R.: Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining (SDM), pp. 443–448 (2007). doi:10.1137/1.9781611972771.42
13. Page, E. S.: Continuous Inspection Schemes. Biometrika **41**(1–2), 100–115 (1954). doi:10.1093/biomet/41.1-2.100
14. Raab, C., Heusinger, M., Schleif, F.-M.: Reactive Soft Prototype Computing for Concept Drift Streams. Neurocomputing **416**, 340–351 (2020). doi:10.1016/j.neucom.2020.01.119
15. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al.: Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences **114**(13), 3521–3526 (2017). doi:10.1073/pnas.1611835114
16. Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Fei-Fei, L., Savarese, S.: Social LSTM: Human trajectory prediction in crowded spaces. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 961–971 (2016). doi:10.1109/CVPR.2016.110
17. Salzmann, T., Ivanovic, B., Chakravarty, P., Pavone, M.: Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In: European Conference on Computer Vision (ECCV) (2020). arXiv:2001.03093

18. Yuan, Y., Weng, X., Ou, Y., Kitani, K.: AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting. In: IEEE/CVF International Conference on Computer Vision (ICCV) (2021). arXiv:2103.14023
19. Gu, T., Chen, G., Li, J., Lin, C., Rao, Y., Zhou, J., Lu, J.: Stochastic Trajectory Prediction via Motion Indeterminacy Diffusion. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2022). arXiv:2203.13777